

Sales & Revenue Intelligence Dashboard

Project Report

Dataset: Sample - Superstore | Tools: Python, pandas, statsmodels, matplotlib

1. Abstract

Retail businesses generate large volumes of transactional sales data, yet conventional sales dashboards are purely descriptive - they report historical revenue but do not explain profitability, forecast demand, detect anomalies, or recommend actions, and they fail on messy real-world data. This project presents an automated, end-to-end Sales & Revenue Intelligence pipeline built in Python that cleans raw sales data, identifies profit-draining products, forecasts future demand using a back-tested model, detects anomalous sales days, and produces actionable business recommendations along with an executive dashboard. It was validated on the real Sample - Superstore dataset (~9,994 transactions, 2015-2018) and uncovered loss-making bestsellers (Tables and Bookcases) that revenue-only dashboards hide.

2. Introduction

Sales data is one of the most valuable assets a company owns, but raw data alone provides no insight. Most dashboards stop at showing past totals and top-selling products by revenue - which is misleading, because a product can sell heavily yet lose money due to deep discounting. Businesses also need to look forward (forecasting) and be alerted to unusual events. This project rebuilds the sales dashboard as an intelligent analytics system that goes from raw data all the way to business recommendations.

3. Problem Statement

Conventional sales dashboards are purely descriptive - they report historical revenue but do not analyze profitability, forecast demand, detect anomalies, or recommend actions, and they assume clean data. There is a need for an automated analytics system that cleans raw sales data, identifies profit-draining products, forecasts demand reliably, flags anomalies, and produces actionable business recommendations.

4. Objectives

1. Clean and validate messy real-world sales data automatically (with an audit log).
2. Analyze profit and margin (not just revenue) to expose loss-making products.
3. Forecast the next 6 months of sales using a back-tested, accuracy-validated model.
4. Detect anomalous sales days automatically.
5. Generate clear, quantified business recommendations.

6. Produce an executive dashboard and a written report.

5. Existing System

How it works: Traditional sales reporting is done manually in Excel or basic BI tools - data is exported, pivot tables and bar charts are built by hand, products are ranked by revenue, and a human eyeballs static charts of past totals.

Limitations:

- Descriptive only ("what happened") - no forecasting or recommendations.
- Focuses on revenue (a vanity metric) - hides products that lose money.
- No automatic anomaly detection.
- Assumes clean data - breaks on nulls, duplicates, and errors.
- Manual and time-consuming; not reproducible.

6. Proposed System

An automated, end-to-end Python pipeline with six modules:

Module	Function
Data Cleaning	Fixes blank rows, duplicates, bad dates, outliers - logs every fix
Profitability Analysis	Computes profit & margin; finds loss-making bestsellers
Forecasting	Predicts next 6 months using a back-tested best model
Anomaly Detection	Automatically flags unusual sales days
Recommendation Engine	Converts findings into business actions
Reporting	Builds an executive dashboard and a written report

7. System Architecture / Workflow

Raw Sales CSV (real Superstore data)

|

- ```
[1] CLEAN -> fix blanks, duplicates, bad dates, outliers
[2] PROFIT -> margin analysis + find loss-making products
[3] FORECAST -> back-test 3 models, pick best, predict 6 months
[4] ANOMALY -> flag unusual sales days (3-sigma rule)
[5] RECOMMEND -> generate business actions
```

[6] REPORT -> dashboard.png + insights\_report.md

|

Actionable business decisions

## 8. Methodology / Modules

7. Data Cleaning (FIX #3): standardize columns, parse dates (drop invalid/blank), normalize text, remove duplicates, drop impossible quantities, cap price outliers.
8. Profitability (FIX #2): group by category/sub-category, compute profit & margin, identify high-revenue but negative-profit products, analyze discount impact.
9. Forecasting (FIX #1a): aggregate to monthly sales, back-test three models, pick the most accurate, retrain on all data, forecast 6 months ahead.
10. Anomaly Detection (FIX #1b): compute daily sales, apply a rolling mean +/- 3-sigma band, flag days outside it.
11. Recommendations (FIX #1c): rule-based logic turns the numbers into actions.
12. Reporting: generate a 4-panel dashboard and a markdown insights report.

## 9. Algorithms Used (and Why)

| Technique                      | Used For             | Why                                             |
|--------------------------------|----------------------|-------------------------------------------------|
| Date coercion (errors=coerce)  | Cleaning             | Detect & drop invalid/blank dates               |
| IQR (Interquartile Range) rule | Outlier removal      | Robust to extreme fat-finger values             |
| Group-by aggregation           | Profitability        | Standard way to summarize grouped data          |
| Median threshold               | Loss-maker detection | Cleanly splits high- vs low-revenue products    |
| Holt-Winters Exp. Smoothing    | Forecasting          | Models level + trend + seasonality (Q4 peak)    |
| SARIMAX (Seasonal ARIMA)       | Forecasting          | Strong statistical model for seasonal series    |
| Log transform                  | Forecasting          | Handles multiplicative (percentage) seasonality |
| Walk-forward back-testing      | Model selection      | Measures honest out-of-sample accuracy          |
| MAPE (Mean Absolute % Error)   | Model scoring        | Interpretable metric to compare models          |
| Rolling mean +/- 3-sigma       | Anomaly detection    | Simple, explainable outlier test                |
| Rule-based logic               | Recommendations      | Converts numbers into business actions          |

## 10. Dataset Description

- Name: Sample - Superstore (a widely used public retail dataset, originally a Tableau sample).
- Source: Public GitHub mirror (leonism/sample-superstore), auto-downloaded by the code.
- Size: 10,800 raw rows -> 9,994 clean transactions after cleaning.
- Period: 2015-2018 (US retail).
- Key columns: Order Date, Region, Category, Sub-Category, Sales, Quantity, Discount, Profit.
- Categories: Furniture, Office Supplies, Technology (17 sub-categories).

## 11. Technologies Used

- Language: Python 3
- Libraries: pandas & numpy (data wrangling), matplotlib (visualization), statsmodels (Holt-Winters & SARIMAX forecasting)
- Tools: Jupyter Notebook, Git/GitHub

## 12. Implementation

- Single-file version: sales\_dashboard.py - the whole pipeline in one runnable file.
- Modular version: main.py + src/ (pipeline.py, forecasting.py, superstore\_loader.py, data\_generator.py) - production-style structure.
- Notebook: notebooks/01\_sales\_analysis.ipynb - a narrative walkthrough.

## 13. Results & Output

**Key Performance Indicators: Total Sales \$2,297,201, Total Profit \$286,397, Overall Margin 12.5%.**

Headline finding - Loss-making bestsellers:

| Sub-Category | Sales     | Profit    | Margin |
|--------------|-----------|-----------|--------|
| Tables       | \$206,966 | -\$17,725 | -8.6%  |
| Bookcases    | \$114,880 | -\$3,473  | -3.0%  |

Profit by Category:

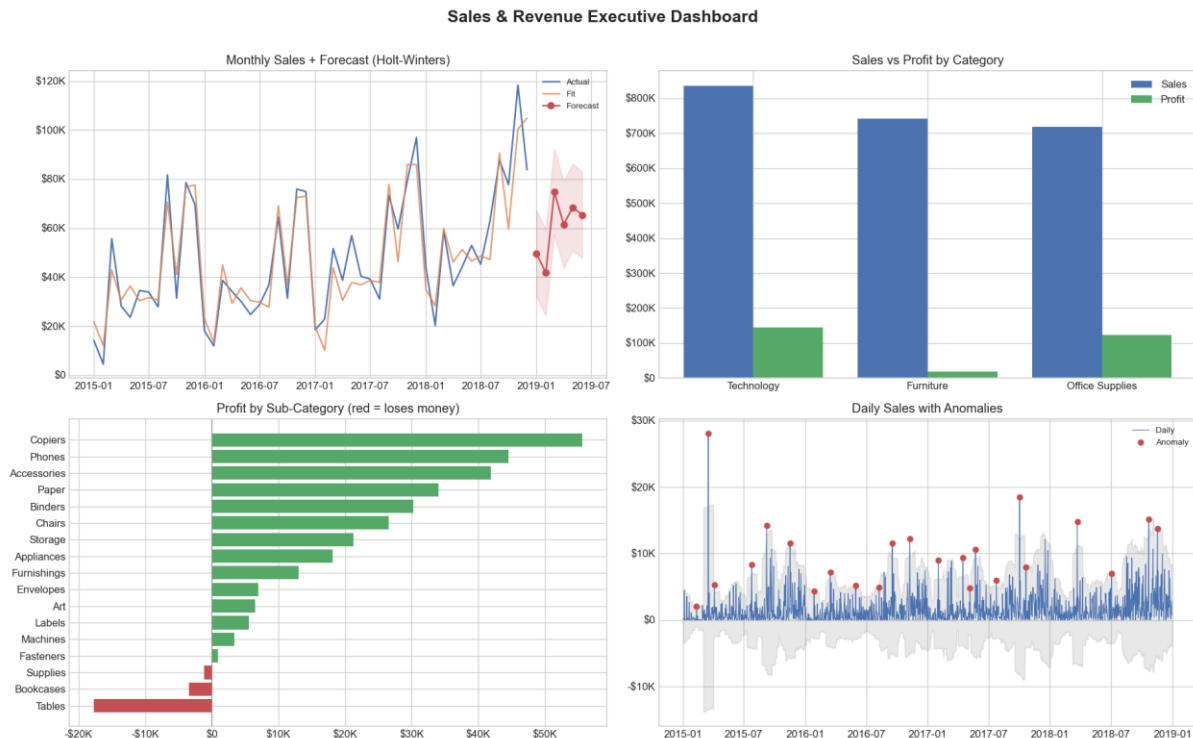
| Category        | Sales     | Profit    | Margin |
|-----------------|-----------|-----------|--------|
| Technology      | \$836,154 | \$145,455 | 17.4%  |
| Furniture       | \$742,000 | \$18,451  | 2.5%   |
| Office Supplies | \$719,047 | \$122,491 | 17.0%  |

Forecast - back-tested model selection (next 6 months = \$361,842):

| Model                    | Out-of-sample MAPE |
|--------------------------|--------------------|
| Holt-Winters (chosen)    | 15.6%              |
| SARIMAX(1,1,1)(1,1,1,12) | 17.4%              |
| Holt-Winters (log)       | 21.3%              |

Other outputs: 23 anomaly days flagged; cleaning retained 9,994 of 10,800 rows (92.5%); an executive dashboard image and a written insights report.

## Executive Dashboard:



## 14. Existing vs Proposed System & Efficiency

| Feature       | Existing System        | Proposed System                       |
|---------------|------------------------|---------------------------------------|
| Data cleaning | Manual / assumed clean | Automated + audited (92.5% recovered) |
| Metric focus  | Revenue (vanity)       | Profit & margin (finds loss-makers)   |
| Future view   | None                   | Back-tested forecast (15.6% error)    |
| Monitoring    | Manual eyeballing      | Automatic anomaly detection           |
| Output        | Static charts          | Charts + written recommendations      |
| Effort        | Hours, manual          | One command, seconds                  |

Efficiency: the system runs the full analysis in seconds, is fully reproducible, achieves a 15.6% out-of-sample forecast error, and automatically surfaced ~\$21K of annual losses that a revenue-only dashboard would hide.

## 15. Real-World Applications

- Retail & e-commerce: monitor sales and profit; fix discount strategy.
- Supply chain & inventory: use the forecast to plan stock levels.
- Business & marketing analytics: decide which products/categories to push.
- Finance: margin and profitability analysis.
- Operations: anomaly detection for stock-outs, pricing bugs, or fraud.

## 16. Limitations & Future Work

- The forecast is univariate (sales over time only). Future work: a SARIMAX model with exogenous drivers (promotions, holidays) to push below 15.6% MAPE.
- Anomaly detection is statistical, not causal - it flags that a day is unusual, not why.
- A real-time version would use a trailing rolling window for live alerting.

## 17. Conclusion

This project demonstrates a complete data-analyst workflow: it cleans messy real data, questions vanity metrics to find hidden losses, forecasts demand with proper back-tested rigor, monitors for anomalies, and recommends concrete actions. It transforms a passive, backward-looking dashboard into an active decision-support tool - showing the difference between simply plotting data and genuinely analyzing it.

## 18. How to Run

```
pip install pandas numpy matplotlib statsmodels
```

```
python sales_dashboard.py # single-file version
```

```
python main.py # modular version (real Superstore data)
```

Outputs are written to the outputs/ folder (charts + insights report).

## 19. Role / Contribution

As the Data Analyst / Developer, responsibilities included: sourcing and loading the dataset, building the automated data-cleaning pipeline, performing profitability and margin analysis, building and back-testing the forecasting models, implementing anomaly detection, generating the dashboard and report, and writing the business recommendations and documentation.